From APIs to MCPs

3 things to keep in mind
when building an MCP Server

Claudiu Nicola

What is MCP?

Before: N × M integrations

Each AI app × each data source = custom integration

3 AI assistants + 5 data sources
= 15 unique connectors to maintain

Each with its own auth, error handling, formatting and custom logic.

With MCP: N + M

An open standard protocol (Anthropic)

3 assistants + 5 data sources
= 3 clients + 5 servers = 8 integrations

A "USB-C" for AI. Build the server once, works with any agent.

#1 APIs are designed for humans. MCPs are designed for AI.

	REST API (Developers)	MCP (AI Agents)
Discovery	You read the docs once	Schema loaded on every request (tokens!)
Composability	You combine small endpoints	Multi-step tool calls = slow and expensive
Flexibility	More options = better	Complexity = hallucinations

Practical Example: Order Tracking

API wrapper: get_user() → list_orders() → get_status() = 3 round-trips

Good MCP: track_order(email) → single tool, internal orchestration

#2 Think in Outcomes, not Operations

v1: API Wrapper

Multiples tools

```
list_calendars()
list_calendar_events(calendar_id)
retrieve_timezone(calendar_id)
retrieve_free_busy_slots(...)
```

"Find free slot for Alice, Bob and Carol" required a chain of **4-6 tool calls**

v2: Outcome-Oriented

A single tool:

```
find_meeting_slot(attendees, duration,
date_range)
```

"Find free slot for Alice, Bob and Carol" → required 1 tool call

#3 Control the token budget

Tool definitions + intermediate results = full context window, slow agent, high costs

Progressive Disclosure

Don't load all tools upfront.

Expose a search_tools() endpoint or let the agent explore a filesystem to discover only what it needs.

150K → 2K tokens
(98.7% reduction)

Filter in code, not in context

The agent writes code that filters results.

10,000 rows → 5 relevant

Loops, conditionals, error handling in execution env, not in the LLM.

Set explicit limits

Goose (Block): files > 400KB → error with actionable message.

Options: truncate, paginate, or throw an explicit error.

The LLM is smart enough to use **`sed/head/tail`**

Source: Anthropic Engineering — "Code Execution with MCP" · Block Engineering

Things to Avoid

Thin wrapper over API

```
GET /users → get_user()
GET /orders → list_orders()
```

If your MCP is a 1:1 copy of the API, it adds no value. The LLM can generate REST call code on its own.

Data dump in context

100+ tool definitions upfront + raw responses with thousands of rows.

Full context window = slow agent, high costs, more frequent hallucinations.

"LLMs can handle API docs just fine" — valid, but...

- True for 1 simple API with good docs. The model generates code directly.
- At scale (10+ systems, multiple agents) you need standardized discovery, outcome-oriented tools, and token control.

Bottom line: A poorly built MCP is worse than no MCP. But a well-designed MCP beats any API doc.

Recap

#1 **APIs = humans · MCPs = AI**
Don't wrap APIs. Design for agents.

#2 **Outcomes > Operations**
One tool, one result. Don't mirror endpoints.

#3 **Control your tokens**
Progressive disclosure, filter in code, explicit limits.

MCP is an interface for AI. Build it like one.